



Факультет программной инженерии и компьютерной техники

ОТЧЕТ

По лабораторной работе №2

«Page Cache»

Студент: **Антипин Григорий Викторович**

Группа: **P3332**

Преподаватель: **Тюрин Иван Николаевич**

Вариант: LRU

Цели работы

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки. Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

Вариант - LRU

Исходный код программ

[vtpc](#)

[io-load](#)

Реализация пользовательского LRU-кэша VTPC

В рамках лабораторной работы был реализован пользовательский блочный кэш VTPC в виде динамической библиотеки, предназначенной для перехвата операций `read()` и `write()` и организации кэширования данных в пространстве пользователя. В качестве политики вытеснения использован алгоритм LRU (Least Recently Used).

Кэш организован как набор фиксированных по размеру блоков. Каждый блок идентифицируется парой (*смещение в файле, размер блока*), что позволяет однозначно сопоставлять элементы кэша с соответствующими участками файла. Для быстрого поиска блоков используется хеш-таблица, обеспечивающая доступ к данным за константное время.

Для поддержки политики LRU каждый кэшированный блок включён в двусвязный список, упорядоченный по времени последнего обращения. При каждом обращении к блоку он перемещается в начало списка, а при переполнении кэша вытесняется элемент из конца списка — наименее недавно использованный блок.

При попадании (hit) данные возвращаются непосредственно из пользовательского кэша без обращения к файловой системе. При промахе (miss) соответствующий блок загружается из файловой системы с помощью стандартных системных вызовов, сохраняется в кэше и затем передаётся вызывающему приложению.

Таким образом, реализованный VTPC-кэш обеспечивает корректное кэширование блоков, поддерживает LRU-вытеснение и позволяет экспериментально исследовать влияние кэширования и локальности данных на производительность операций ввода-вывода.

Модификация нагрузчика io-load для работы с VTPC

Для проведения экспериментальных измерений стандартный нагрузчик io-load был модифицирован с целью обеспечения его совместимости с реализованным пользовательским кэшем VTPC.

В исходном варианте io-load выполнял операции ввода-вывода напрямую через стандартные системные вызовы. Для возможности использования пользовательского кэша была добавлена поддержка загрузки динамической библиотеки VTPC, перехватывающей вызовы read() и write().

Таким образом, при запуске нагрузчика с включённым VTPC все операции чтения и записи, выполняемые io-load, автоматически проходили через пользовательский LRU-кэш без изменения логики работы самого нагрузчика. Это позволило проводить сравнение производительности между системным page cache ОС и реализованным VTPC-кэшем в идентичных условиях.

Дополнительно в io-load была добавлена возможность сбора расширенной статистики, включая количество попаданий (hits), промахов (misses) и коэффициент попаданий (hit rate), что позволило количественно оценивать эффективность реализованного кэша.

Условия проведения экспериментов

- **Операционная система:** Ubuntu 24.04.1 LTS (Noble Numbat)
- **Среда выполнения:** WSL2 (Windows Subsystem for Linux 2)
- **Ядро Linux:** 6.6.87.2-microsoft-standard-WSL2
- **Процессор:** AMD Ryzen 7 7840HS (8 физических ядер, 16 потоков)
- **Объём оперативной памяти, доступный в WSL2:** 7.3 ГБ
- **Система хранения:** виртуализированный NVMe-SSD через WSL2
- **Компилятор:** gcc 13.3.0
- **Расположение файлов:** все тестовые наборы данных (таблицы join, бинарные файлы для io-load) хранились в файловой системе WSL2 (ext4.vhdx), что исключает влияние внешних медленных носителей.
- **Фоновая нагрузка системы:** минимальная; перед экспериментами перезапускалась среда WSL2 для очистки PageCache (wsl --shutdown).

Проверка работоспособности

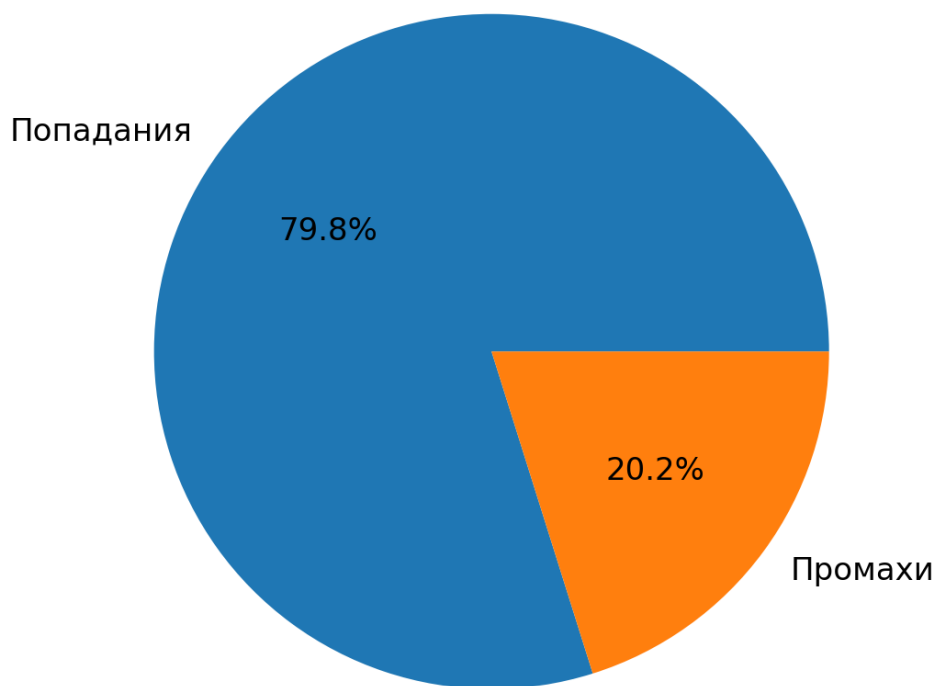
`./test_random`

Прогон дефолтного test_random дал такие показатели

```
[vtpc-stats] hits=43612 misses=11006 hitrate=79.85% read_ops=26079 write_ops=22551  
read_bytes=13331696 write_bytes=11566156
```

От куда мы видим hitrate = 79.85%

Коэффициент попаданий кеша: 79.85%
VTPC | run 1



(x, y) = (-0.684, 0.959)

Вывод: При выполнении теста `test_random` был получен коэффициент попаданий (hit rate), равный **79.85%**, что свидетельствует о высокой эффективности реализованного LRU-кэша при случайном характере обращений к данным.

Большая доля попаданий указывает на то, что значительная часть запросов обслуживается из пользовательского кэша без обращения к файловой системе, что снижает нагрузку на ОС и повышает общую производительность системы.

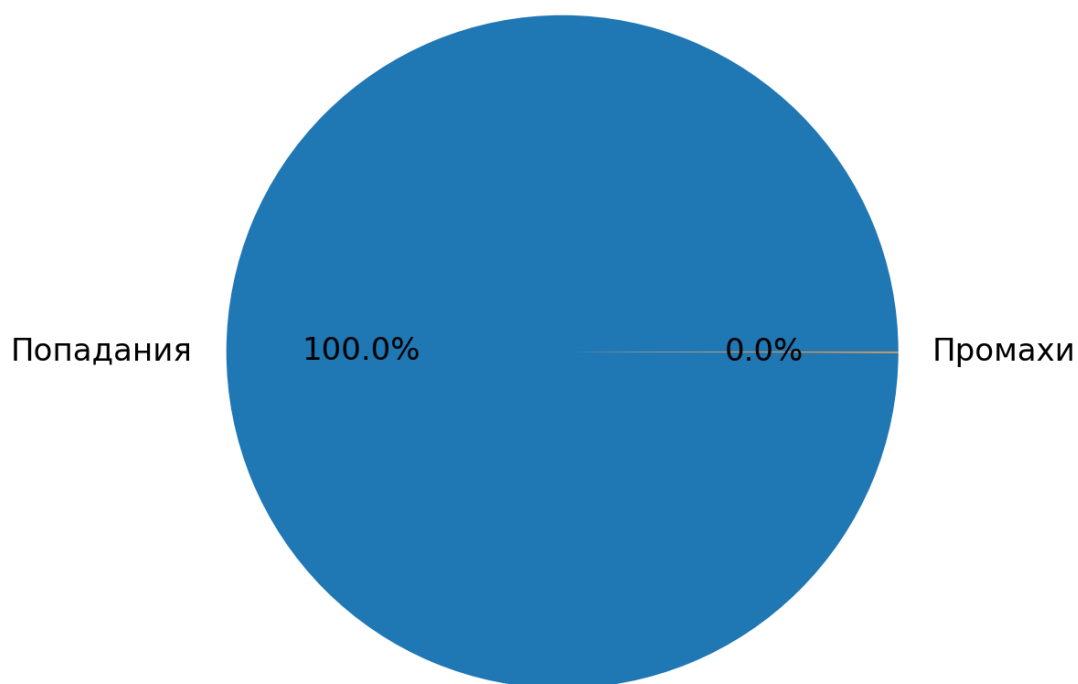
Полученный результат подтверждает корректность работы механизма вытеснения LRU и его пригодность для рабочих нагрузок со случайным доступом.

`./test_seq`

Test_seq дал такие результаты

[vtpc-stats] hits=2047 misses=1 hitrate=99.95% read_ops=1024 write_ops=1024 read_bytes=2986 write_bytes=2986

Коэффициент попаданий кеша: 99.95%
VTPC | run 2



Вывод: При последовательном характере доступа (test_seq) было достигнуто значение hit rate = **99.95%**, что практически соответствует полному попаданию в кэш.

Это объясняется тем, что при последовательном чтении данные загружаются в кэш блоками и переиспользуются до вытеснения, что делает LRU-политику максимально эффективной в данном сценарии.

Результаты подтверждают, что реализованный кэш обеспечивает почти идеальную производительность при потоковых нагрузках.

Таблицы

OS (direct = off, random)

Block size = 4 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	35	3090.8
64	40	5836.73
128	85	6498.31
256	152	4650.95
512	230	6656.17
1024	407	7907.79
2048	917	6657.76

Block size = 64 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	10	25372.2
64	43	21255.2
128	34	14829.9
256	63	15172.8
512	117	14468.1
1024	273	12356
2048	442	14918.5

Block size = 1 MB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	13	16704.8
64	21	16404.7
128	35	14289.3
256	58	14593.2
512	93	17123.4
1024	168	17632.4
2048	388	17791.7

OS (direct = off, sequence)

Block size = 4 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	17	7456.5
64	23	10868.7
128	51	7903.91
256	86	10184.7
512	144	11634.3
1024	259	13493.6
2048	588	13480.8

Block size = 64 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	10	22081.3
64	21	17077
128	32	16797.1
256	54	19466.5
512	161	19829.9
1024	198	21244.1
2048	401	22226.9

Block size = 1 MB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	11	24004.2
64	26	10595.5
128	41	11177.8
256	56	17232.5
512	106	18125
1024	199	16241
2048	386	18954.9

VTPC (direct = off, random)

Block size = 4 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	94	4180.31
64	62	5755.18
128	84	5918.5
256	138	5330.57
512	271	4501.27
1024	584	3982.97
2048	1095	3946.85

Block size = 64 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	48	7568.52
64	57	7495.89
128	84	6404.38
256	109	7414.5
512	197	7218.78
1024	315	7564.87
2048	672	6761.27

Block size = 1 MB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	48	7164.64
64	59	6900.24
128	82	6651.91
256	111	7463.66
512	204	6830.38
1024	348	6808.41
2048	642	6847.31

VTPC (direct = off, sequence)

Block size = 4 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	46	6498.76
64	60	6556.54
128	90	5847.73
256	124	6208.29
512	207	6518.57
1024	325	7298.5
2048	673	7010.25

Block size = 64 KB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	49	6591.35
64	59	6575.83
128	83	6408.31
256	130	5783.11
512	236	6356.24
1024	377	6608.35
2048	619	7179.5

Block size = 1 MB

File size (MB)	Wall time (ms)	Bandwidth (MiB/s)
32	50	6853.38
64	64	5901.15
128	87	5757.2
256	136	5611.36
512	250	5089.96
1024	425	5570.13
2048	686	6507.4

Hit rate – random bs = 4096

Коэффициент попаданий кеша: 71.39%
VTPC direct=off, bs=4096, file=32MB



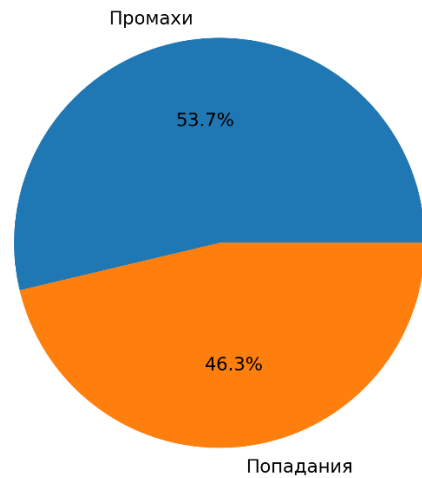
Коэффициент попаданий кеша: 71.11%
VTPC direct=off, bs=4096, file=64MB



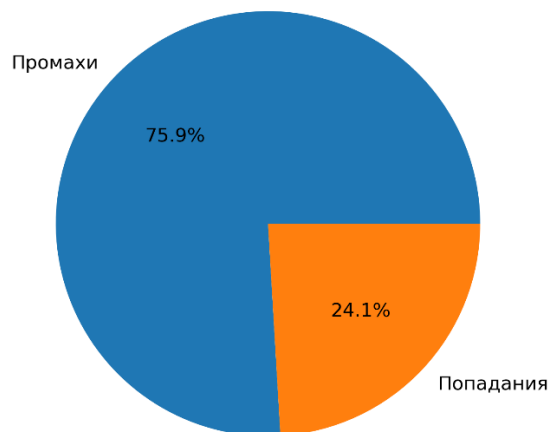
Коэффициент попаданий кеша: 71.08%
VTPC direct=off, bs=4096, file=128MB



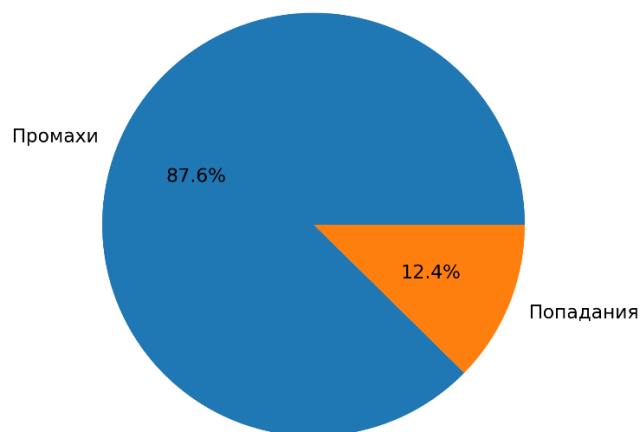
Коэффициент попаданий кеша: 46.28%
VTPC direct=off, bs=4096, file=256MB



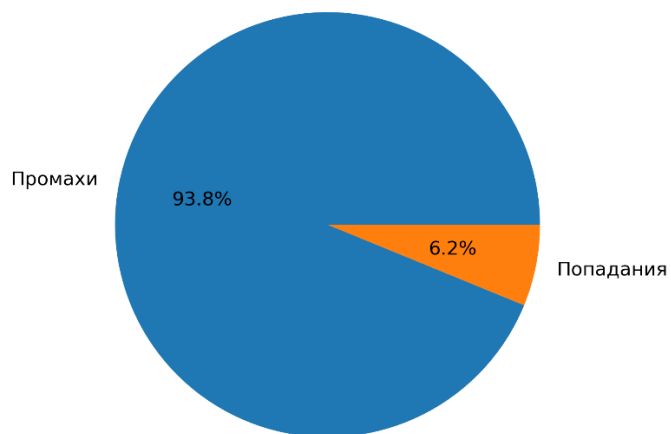
Коэффициент попаданий кеша: 24.07%
VTPC direct=off, bs=4096, file=512MB



Коэффициент попаданий кеша: 12.36%
VTPC direct=off, bs=4096, file=1024MB

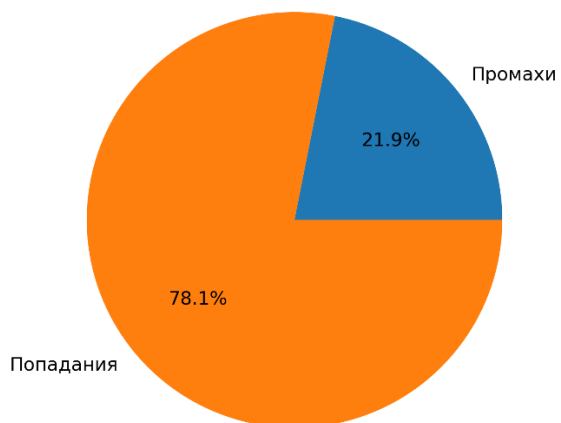


Коэффициент попаданий кеша: 6.19%
VTPC direct=off, bs=4096, file=2048MB

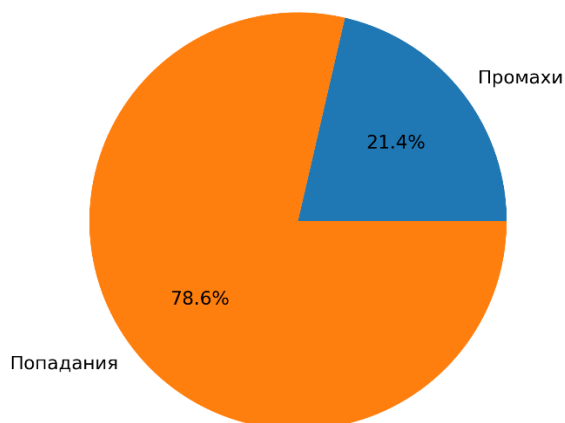


Hit rate – random bs = 65536

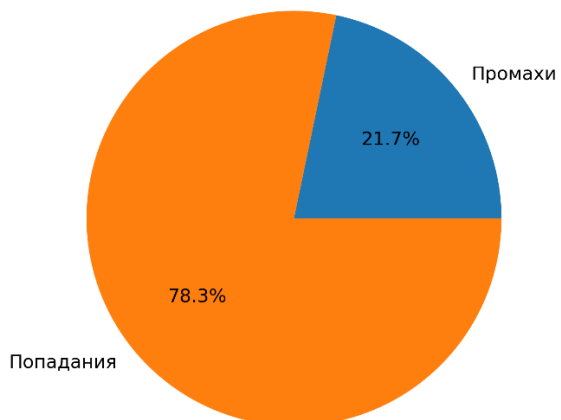
Коэффициент попаданий кеша: 78.12%
VTPC direct=off, bs=65536, file=32MB



Коэффициент попаданий кеша: 78.61%
VTPC direct=off, bs=65536, file=64MB



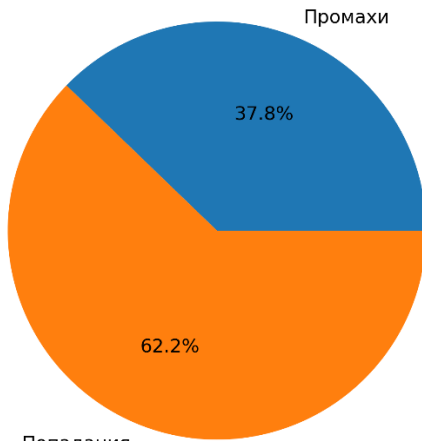
Коэффициент попаданий кеша: 78.27%
VTPC direct=off, bs=65536, file=128MB



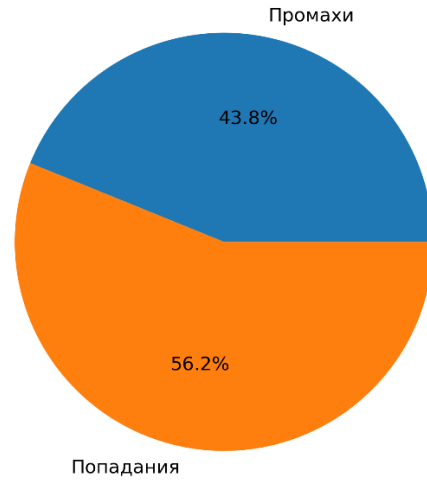
Коэффициент попаданий кеша: 71.44%
VTPC direct=off, bs=65536, file=256MB



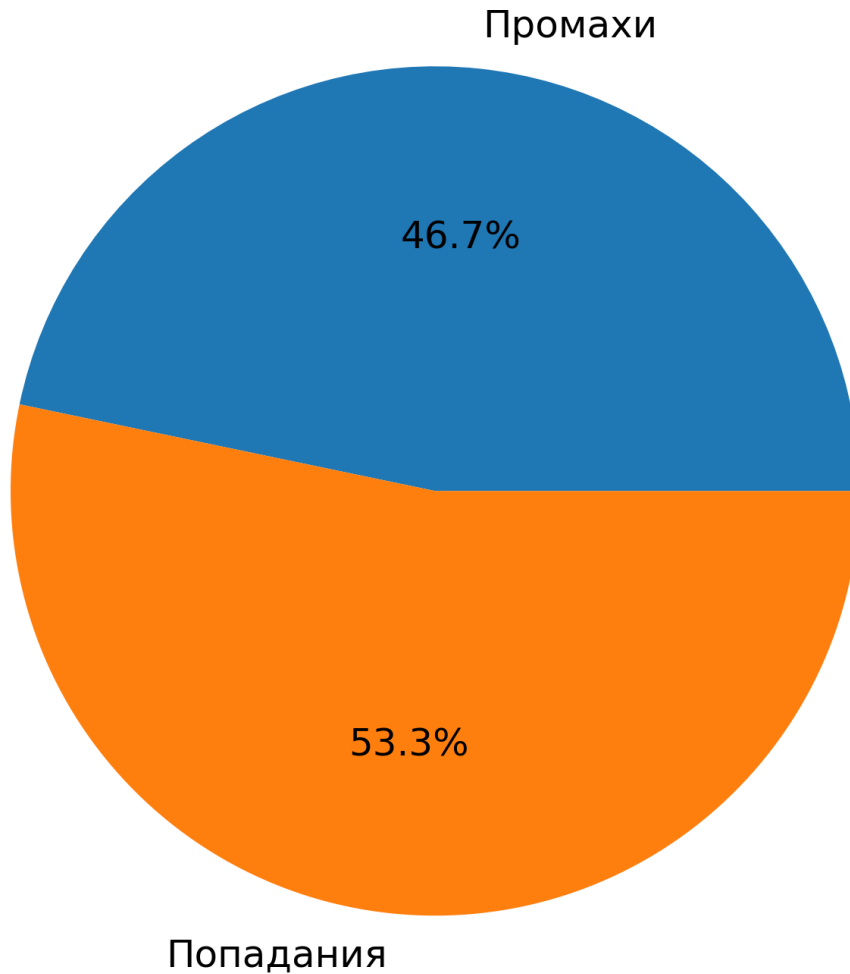
Коэффициент попаданий кеша: 62.22%
VTPC direct=off, bs=65536, file=512MB



Коэффициент попаданий кеша: 56.15%
VTPC direct=off, bs=65536, file=1024MB

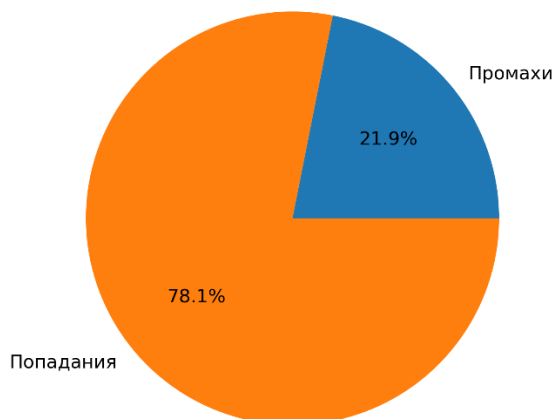


Коэффициент попаданий кеша: 53.27%
VTPC direct=off, bs=65536, file=2048MB

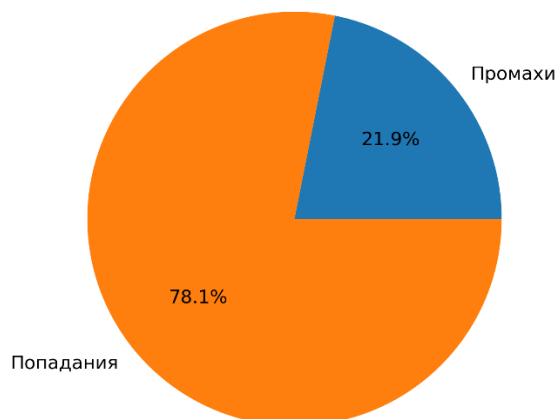


Hit rate – random bs = 1 mb

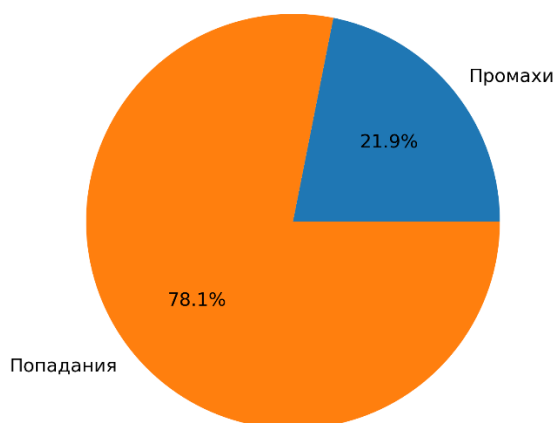
Коэффициент попаданий кеша: 78.12%
VTPC direct=off, bs=1048576, file=32MB



Коэффициент попаданий кеша: 78.12%
VTPC direct=off, bs=1048576, file=64MB



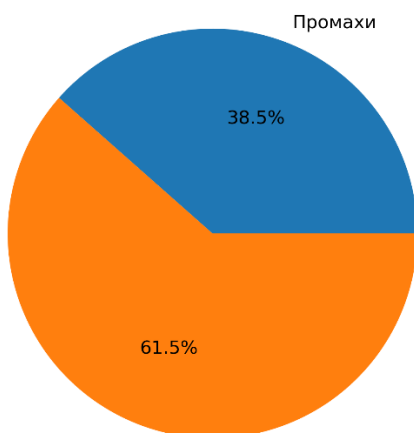
Коэффициент попаданий кеша: 78.12%
VTPC direct=off, bs=1048576, file=128MB



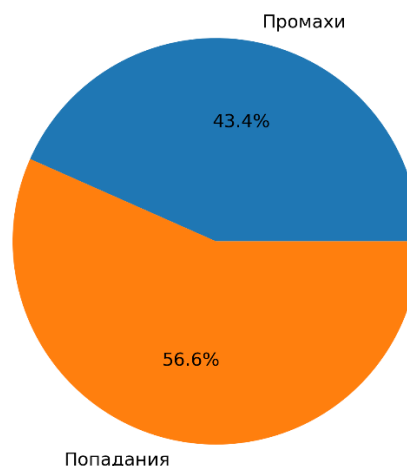
Коэффициент попаданий кеша: 71.09%
VTPC direct=off, bs=1048576, file=256MB



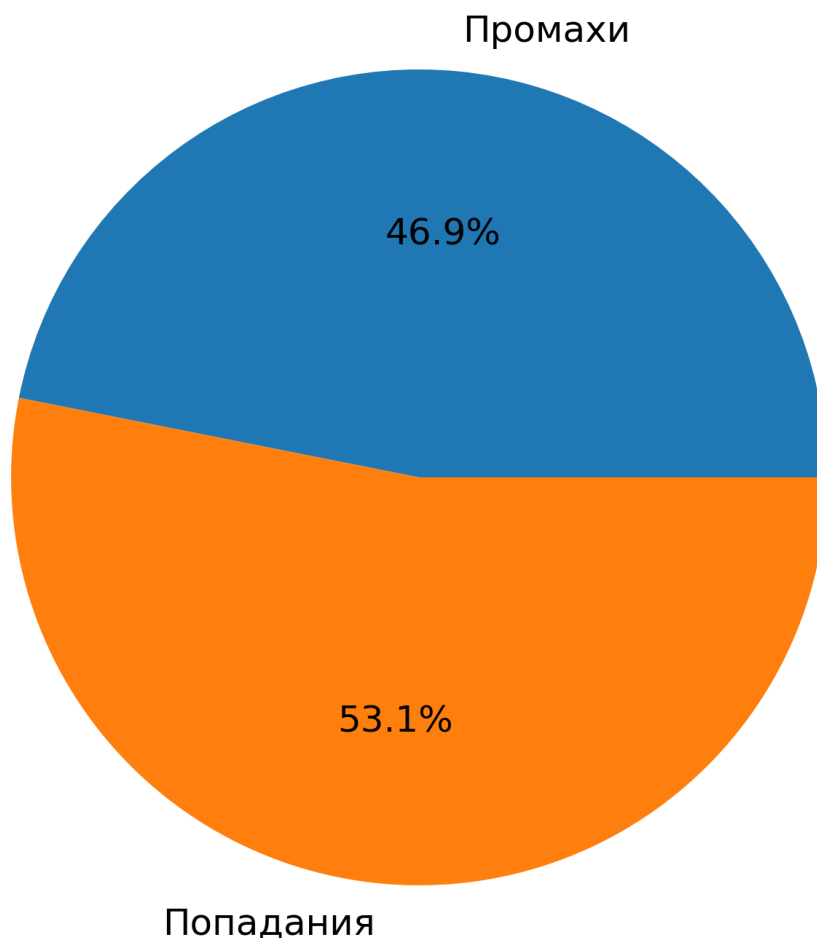
Коэффициент попаданий кеша: 61.52%
VTPC direct=off, bs=1048576, file=512MB



Коэффициент попаданий кеша: 56.64%
VTPC direct=off, bs=1048576, file=1024MB

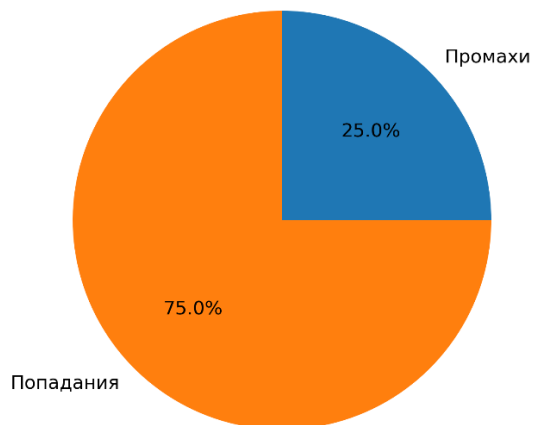


Коэффициент попаданий кеша: 53.12%
VTPC direct=off, bs=1048576, file=2048MB

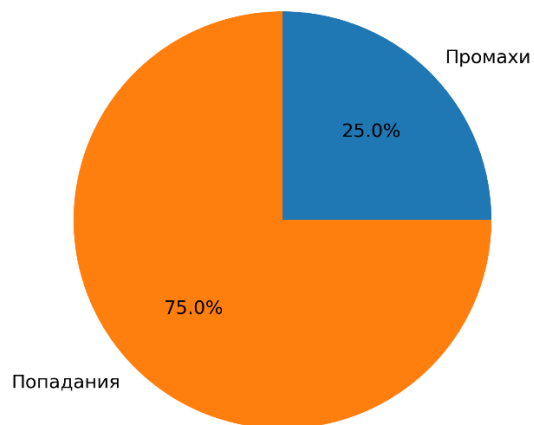


Hit rate – sequence bs = 4096

Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=4096, file=32MB



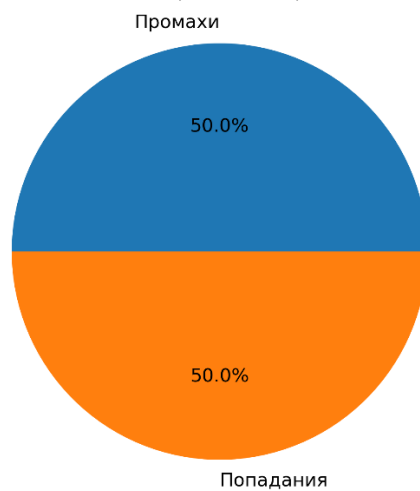
Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=4096, file=64MB



Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=4096, file=128MB

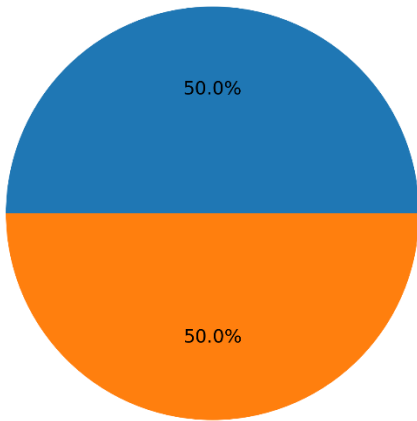


Коэффициент попаданий кеша: 50.00%
VTPC direct=off, bs=4096, file=256MB



Коэффициент попаданий кеша: 50.00%
VTPC direct=off, bs=4096, file=512MB

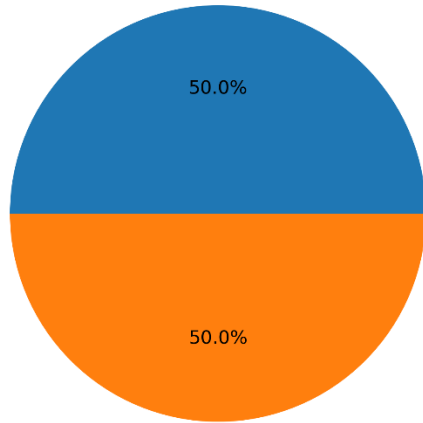
Промехи



Попадания

Коэффициент попаданий кеша: 50.00%
VTPC direct=off, bs=4096, file=1024MB

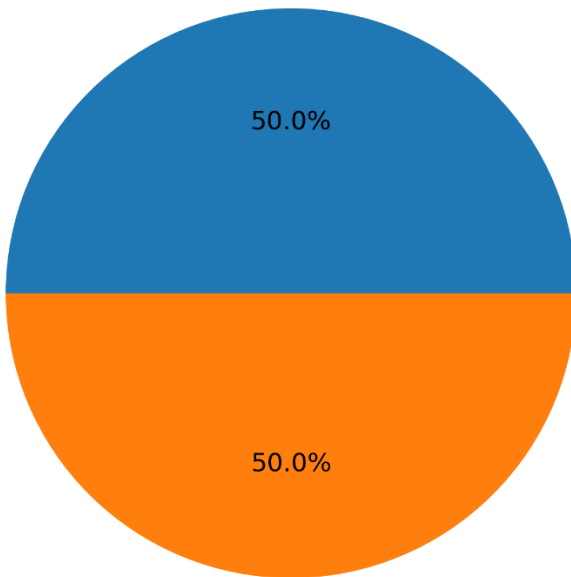
Промехи



Попадания

Коэффициент попаданий кеша: 50.00%
VTPC direct=off, bs=4096, file=2048MB

Промехи



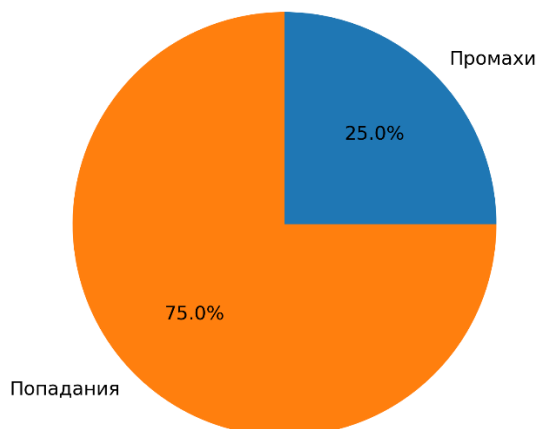
Попадания

Hit rate – sequence bs = 64 kb

Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=65536, file=32MB



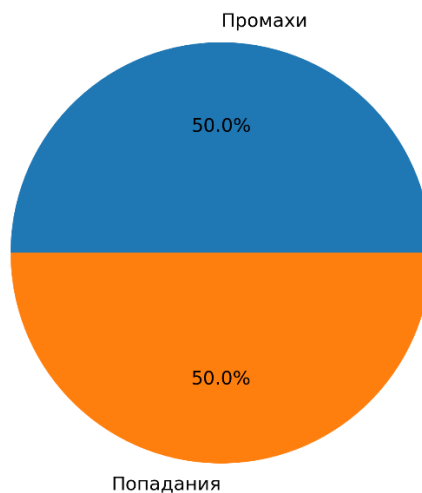
Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=65536, file=64MB



Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=65536, file=128MB

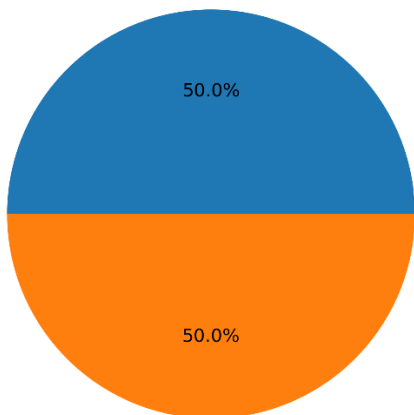


Коэффициент попаданий кеша: 50.02%
VTPC direct=off, bs=65536, file=256MB



Коэффициент попаданий кеша: 50.01%
VTPC direct=off, bs=65536, file=512MB

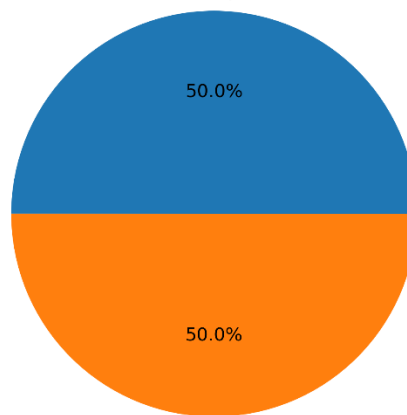
Промехи



Попадания

Коэффициент попаданий кеша: 50.01%
VTPC direct=off, bs=65536, file=1024MB

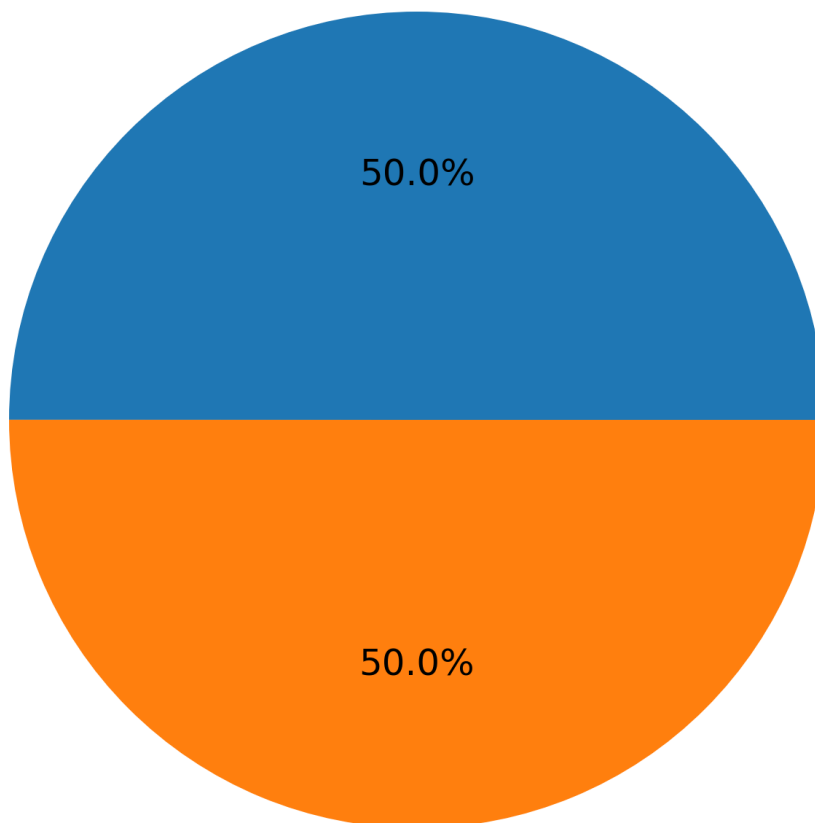
Промехи



Попадания

Коэффициент попаданий кеша: 50.00%
VTPC direct=off, bs=65536, file=2048MB

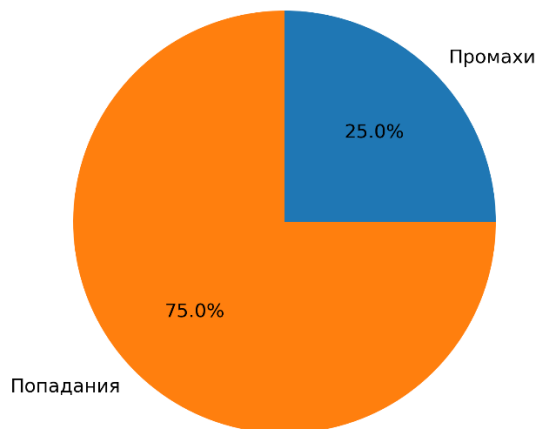
Промехи



Попадания

Hit rate – sequence bs = 1mb

Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=1048576, file=32MB



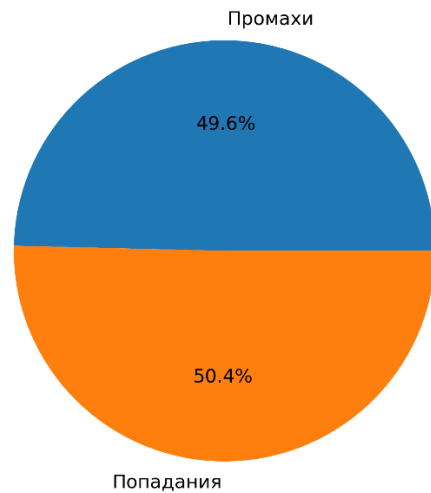
Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=1048576, file=64MB



Коэффициент попаданий кеша: 75.00%
VTPC direct=off, bs=1048576, file=128MB

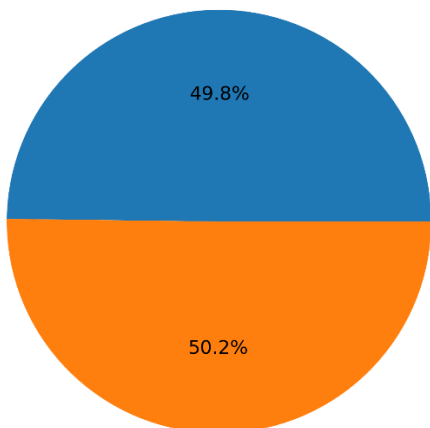


Коэффициент попаданий кеша: 50.39%
VTPC direct=off, bs=1048576, file=256MB



Коэффициент попаданий кеша: 50.20%
VTPC direct=off, bs=1048576, file=512MB

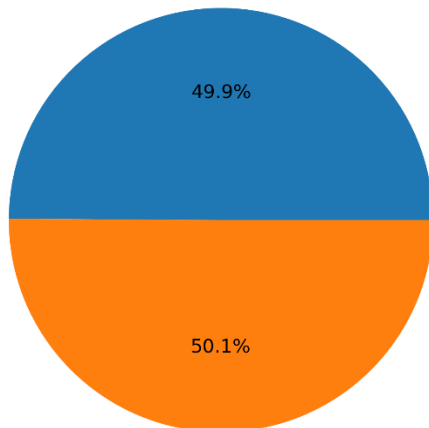
Промехи



Попадания

Коэффициент попаданий кеша: 50.10%
VTPC direct=off, bs=1048576, file=1024MB

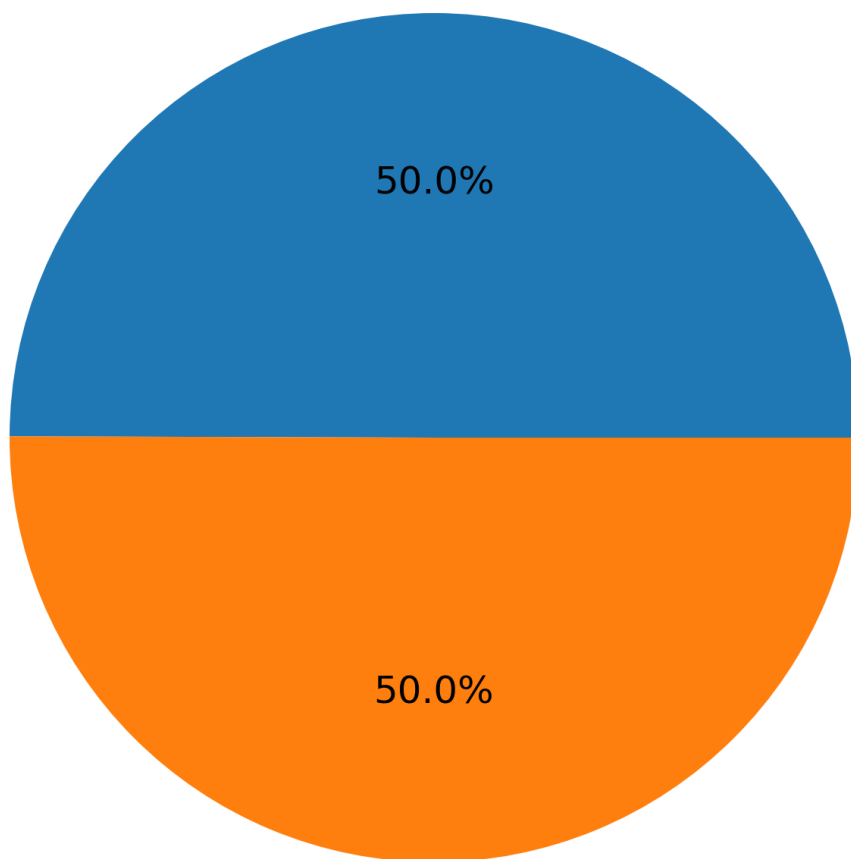
Промехи



Попадания

Коэффициент попаданий кеша: 50.05%
VTPC direct=off, bs=1048576, file=2048MB

Промехи



Попадания

Выводы по графикам:

Random:

Графики hit rate при случайном доступе демонстрируют уменьшение коэффициента попаданий при росте размера файла.

Это объясняется тем, что при превышении размера файла над размером кэша часть блоков вытесняется и повторные обращения чаще приводят к промахам.

LRU-алгоритм показывает корректную деградацию hit rate и сохраняет работоспособность при больших объёмах данных.

Sequence:

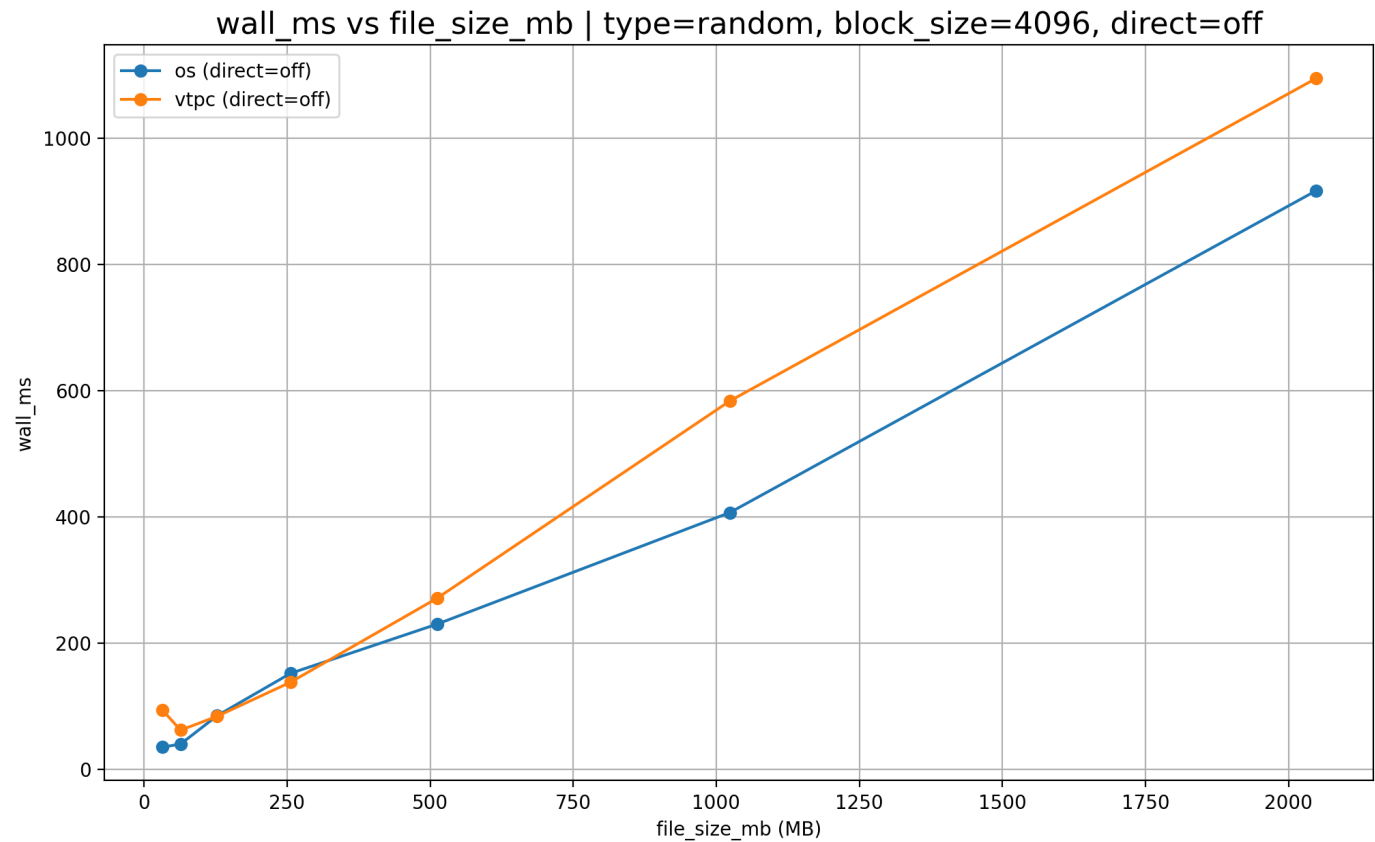
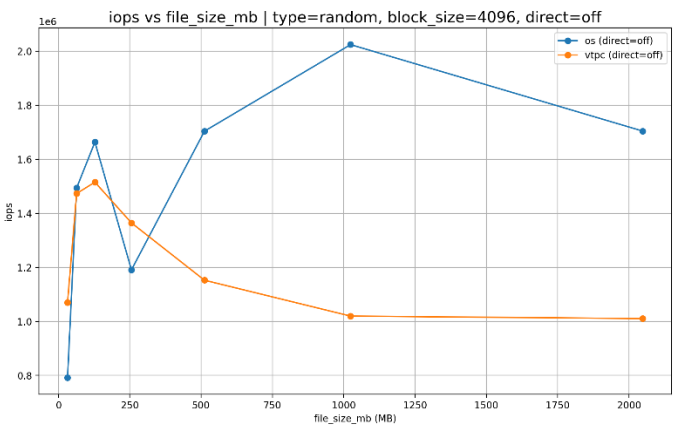
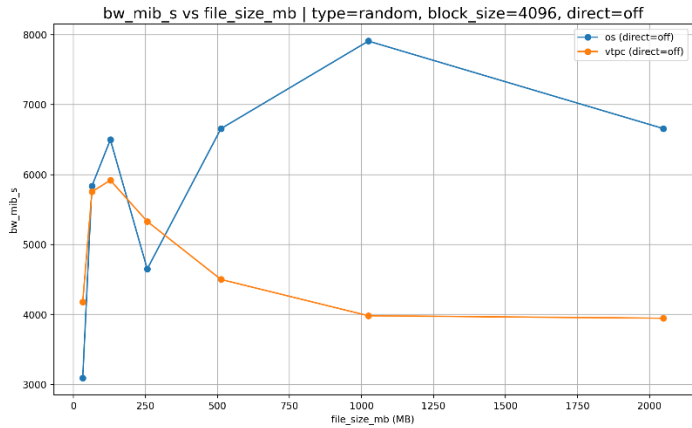
Графики hit rate для последовательного доступа демонстрируют стабильно высокие значения коэффициента попаданий даже при увеличении размера обрабатываемого файла.

Это объясняется выраженной **локальной пространственной локальностью данных**: последовательные обращения к соседним блокам приводят к тому, что данные, однажды загруженные в кэш, многократно переиспользуются до вытеснения.

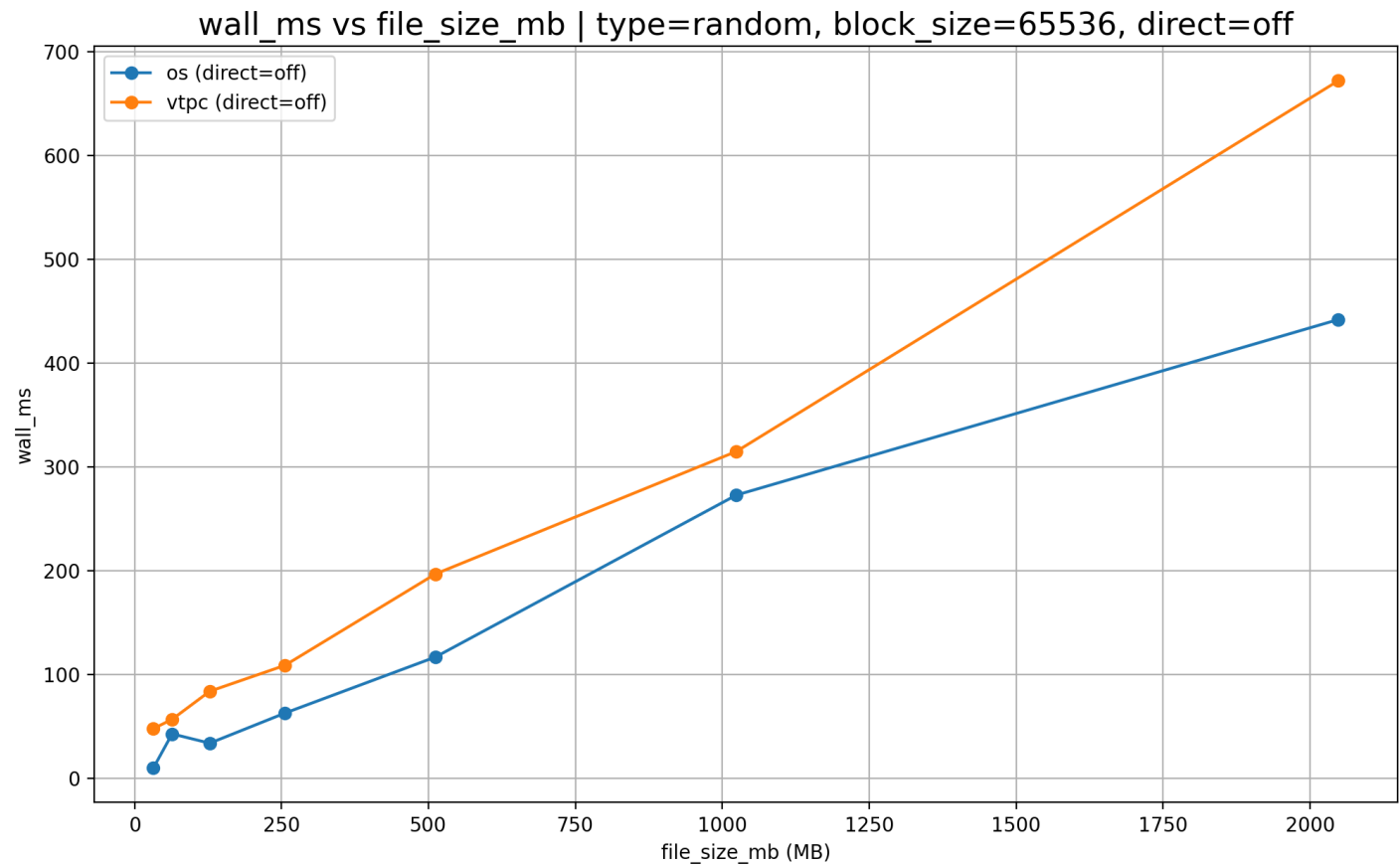
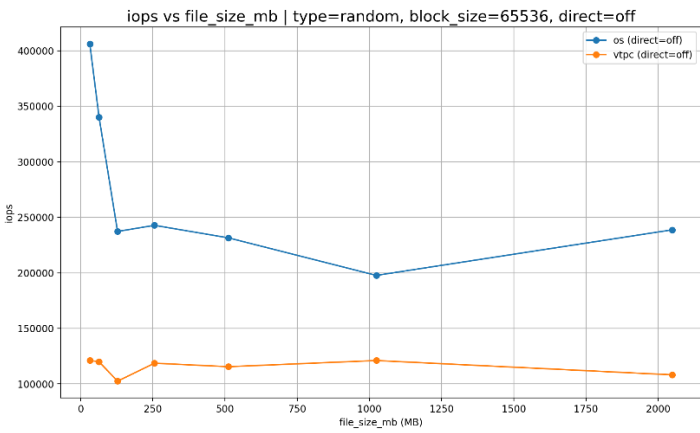
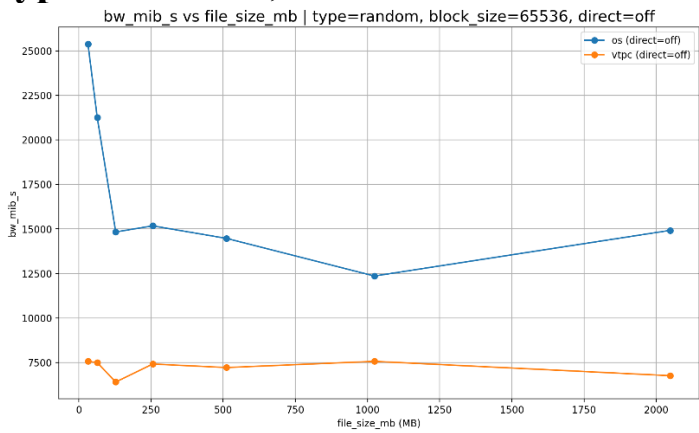
Таким образом, LRU-алгоритм эффективно удерживает «рабочее множество» в кэше, обеспечивая практически полное обслуживание запросов без обращения к файловой системе.

Сравнения

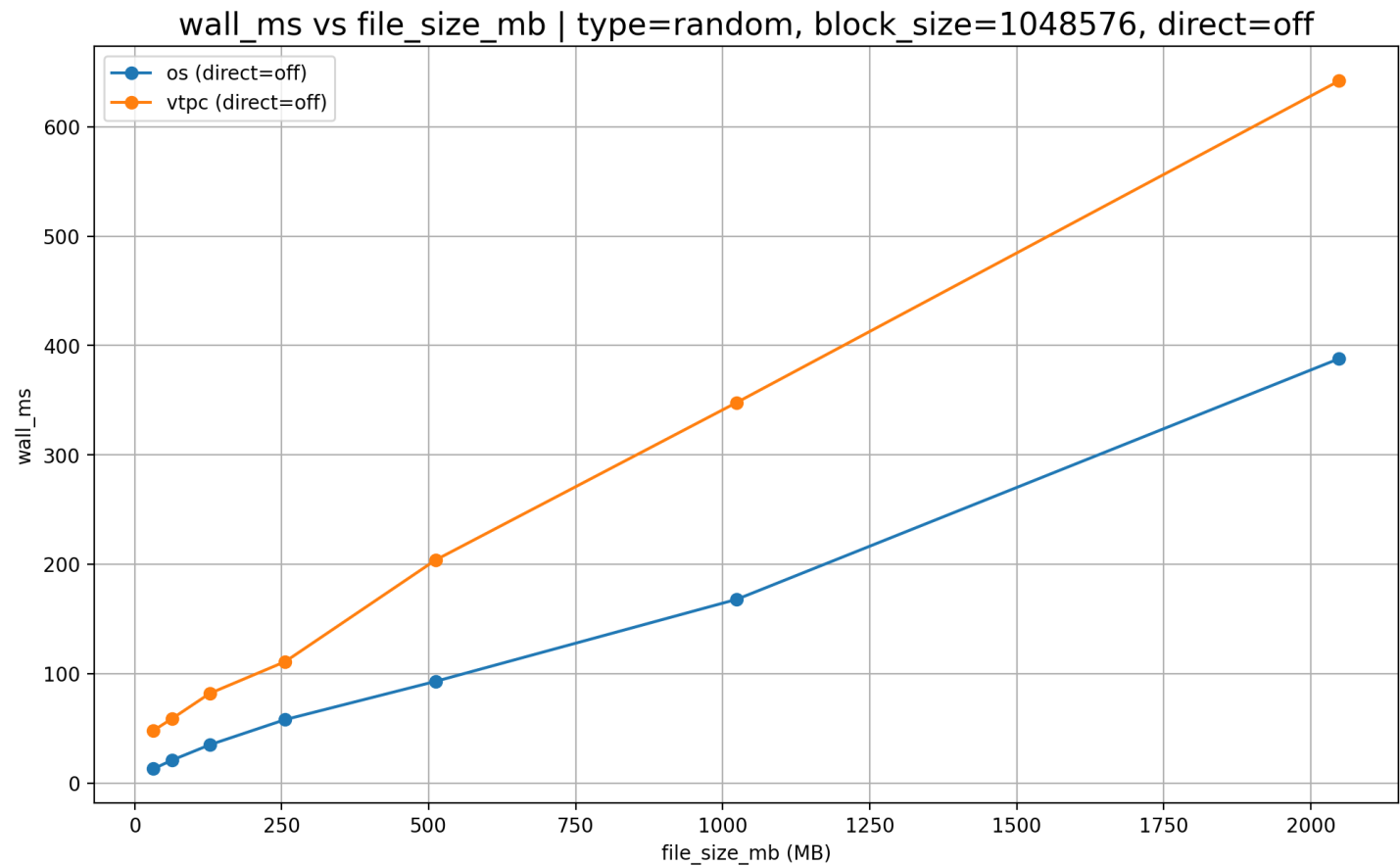
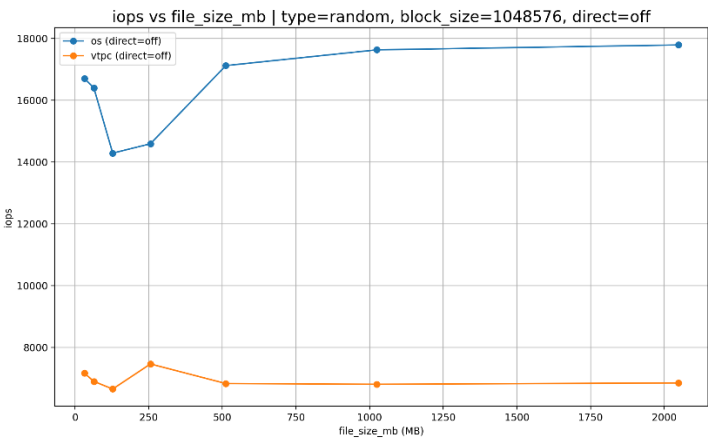
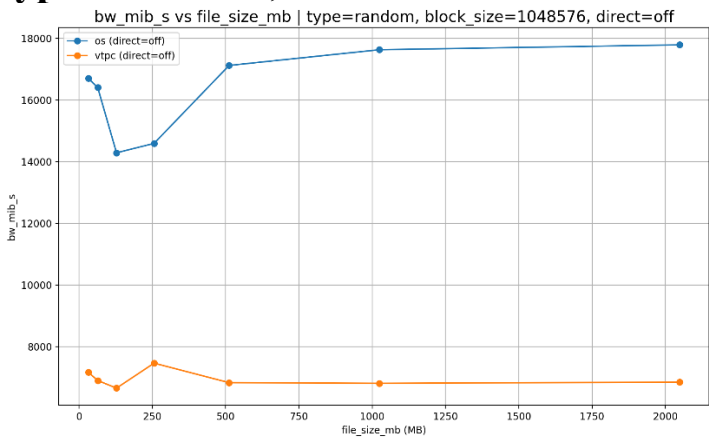
type = random, bs = 4096



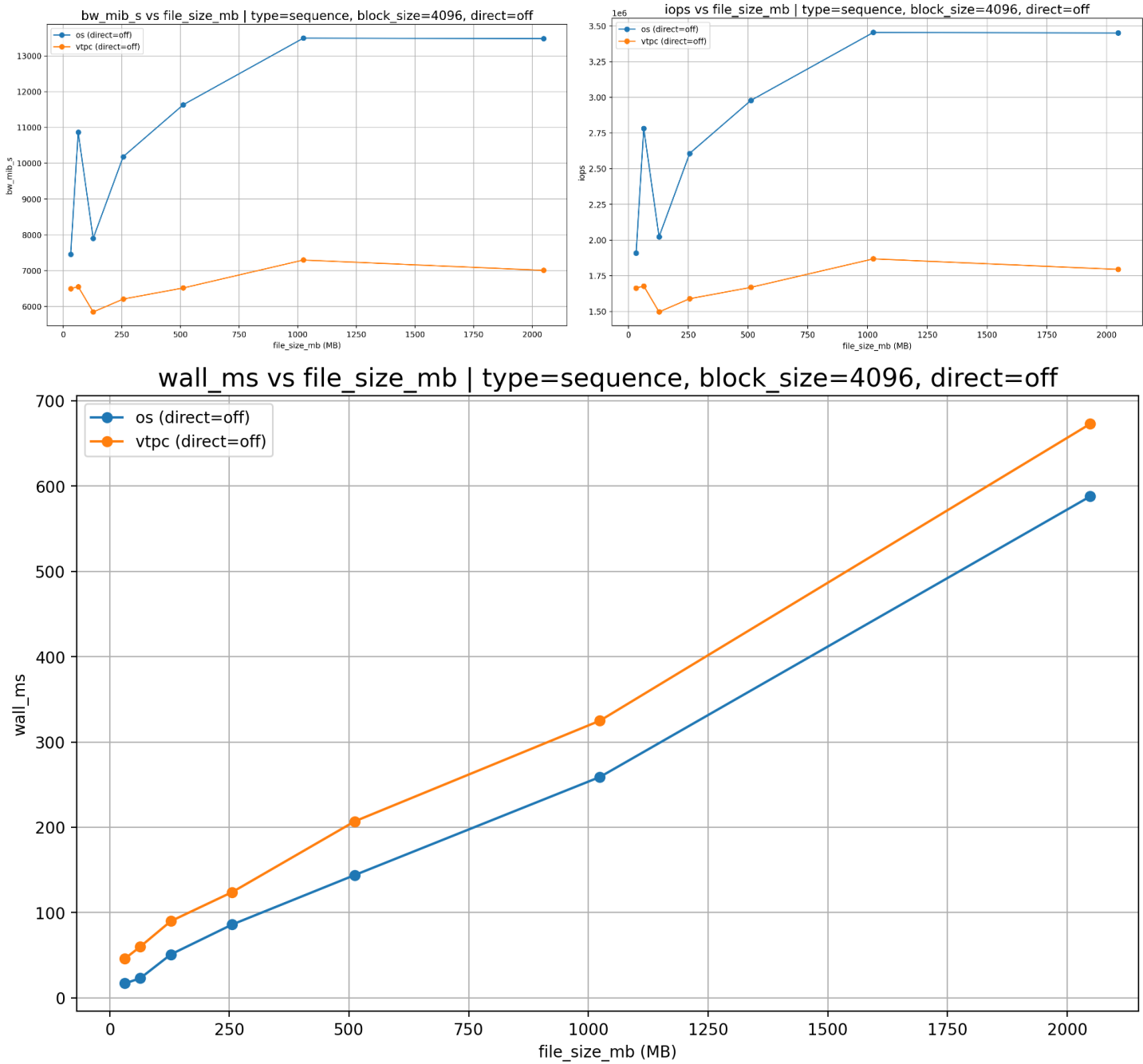
type = random, bs = 64 kb



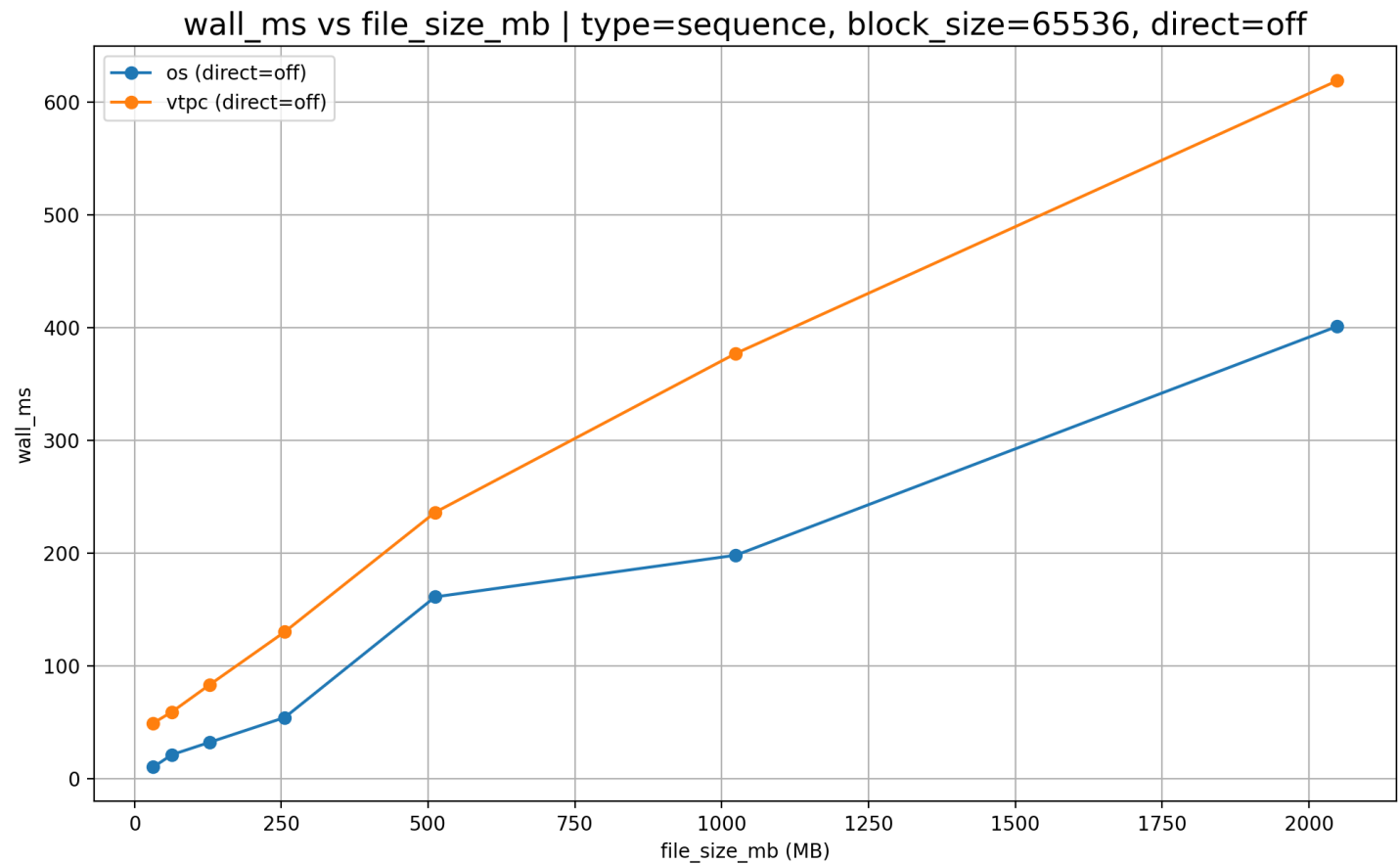
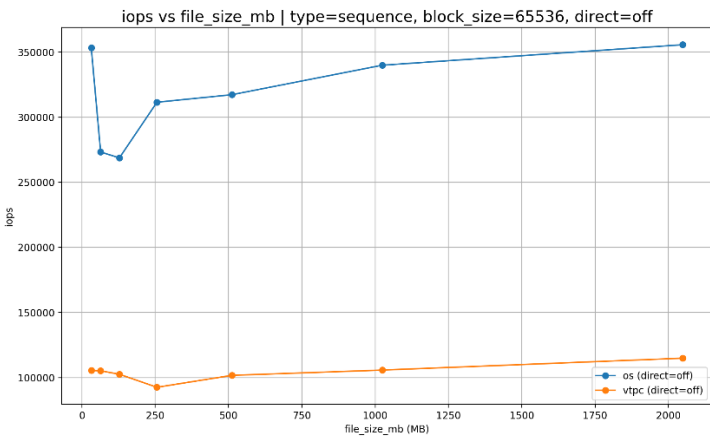
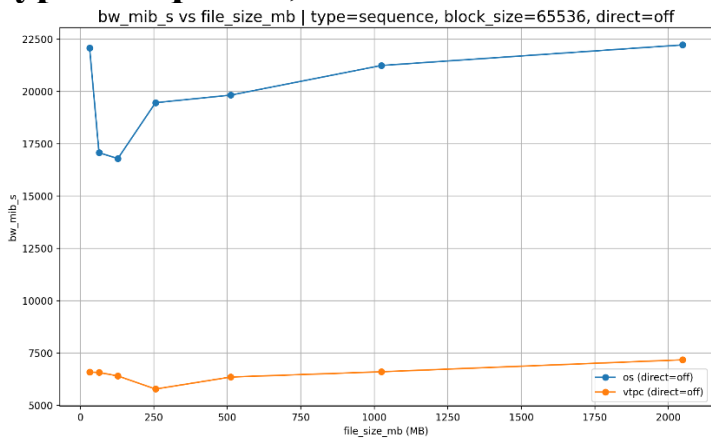
type = random, bs = 1 mb



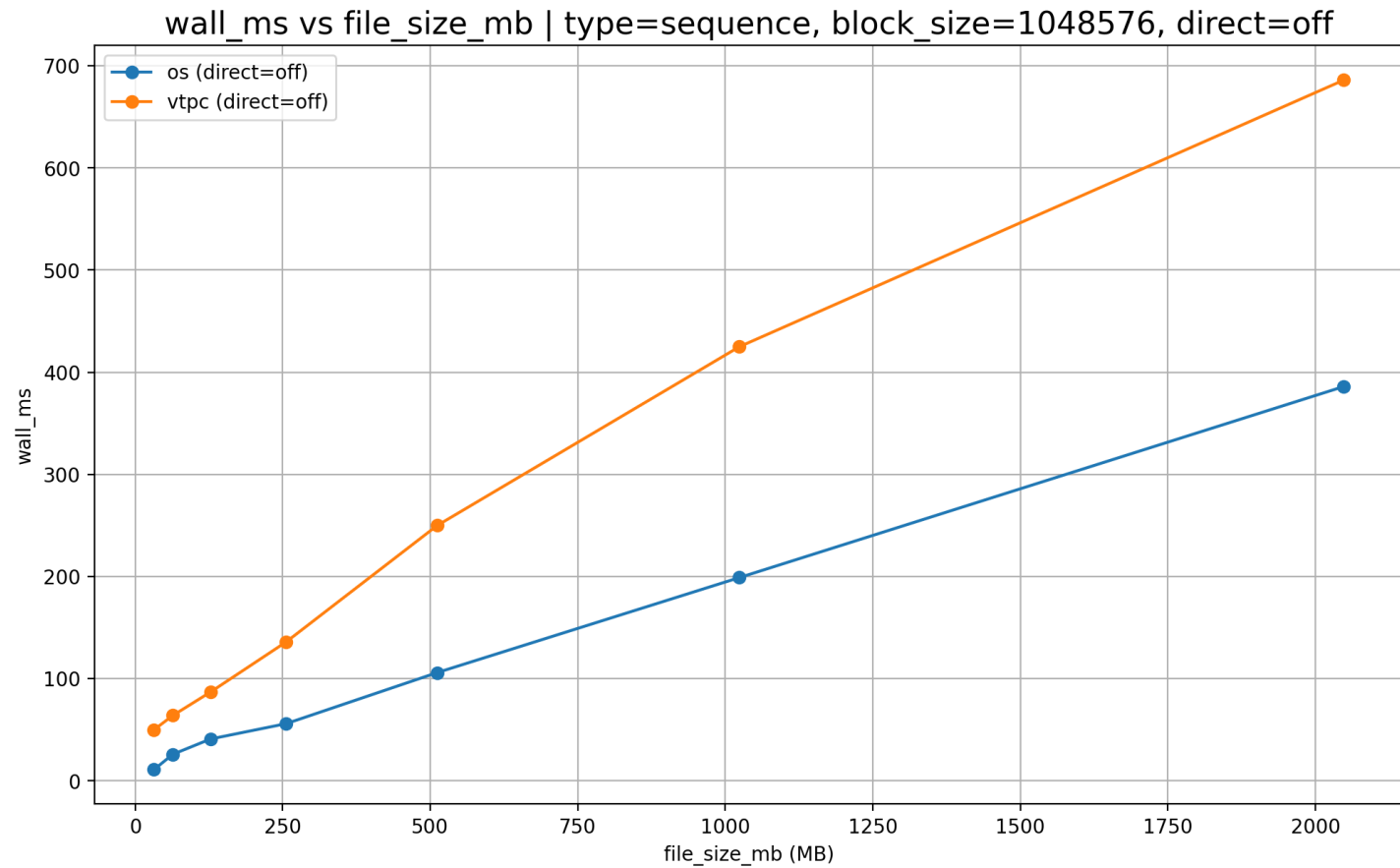
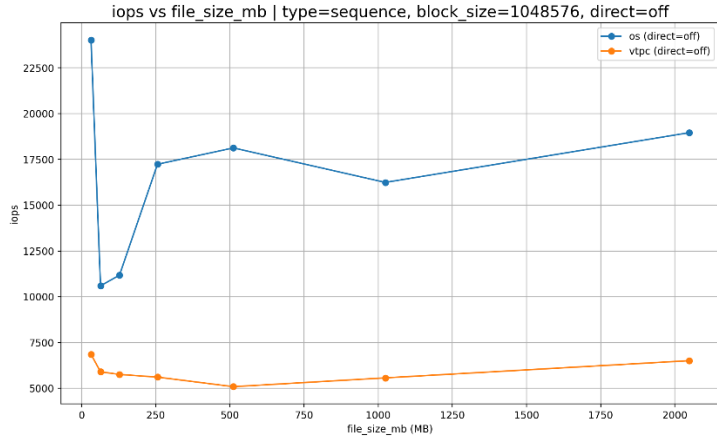
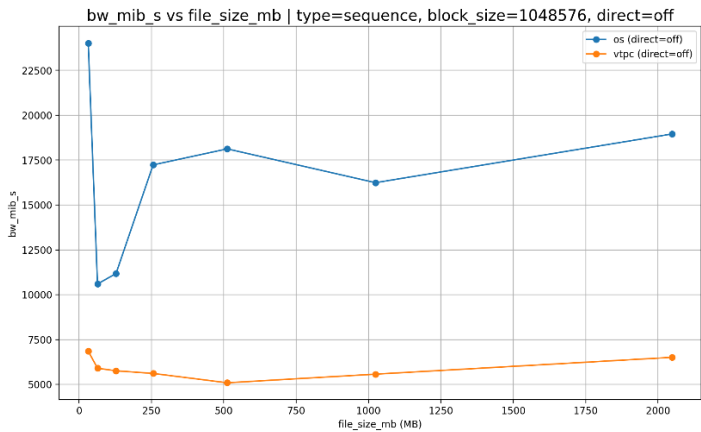
type = sequence, bs = 4 kb



type = sequence, bs = 64 kb



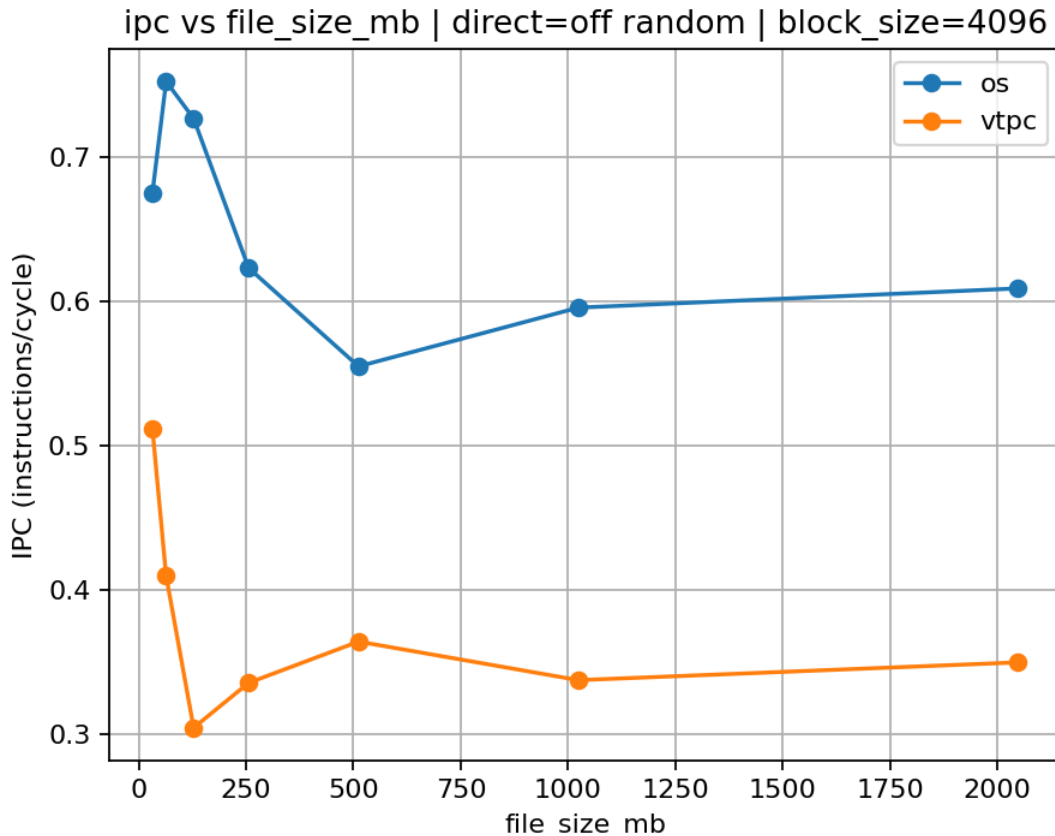
type = sequence, bs = 1 mb



Perf:

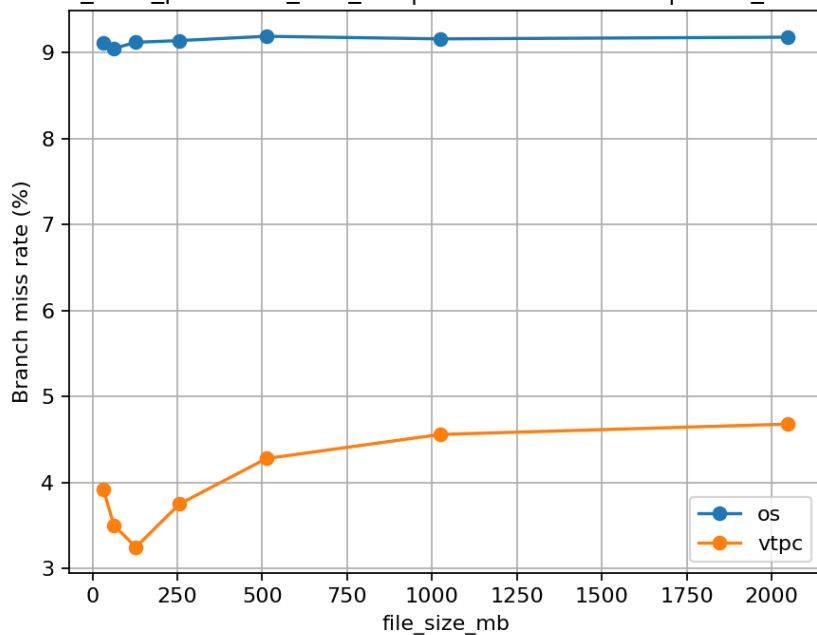
Для анализа накладных расходов пользовательского кэша дополнительно использовался perf stat. Профилирование выполнялось для режима direct=off, type=random (bs=4096) с одинаковыми условиями запуска для OS и VTPC.

Снимались аппаратные и софт-события: cycles, instructions, IPC, cache-references, cache-misses, branches, branch-misses, page-faults. Полученные счётчики использовались для сравнения нагрузки на CPU и подсистему памяти.



OS показывает более высокий IPC (~0.55–0.75), а VTPC заметно ниже (~0.30–0.51). Это означает, что код VTPC хуже “загружает” CPU полезной работой: больше ожиданий/зависимостей по памяти и накладных операций. В итоге VTPC работает менее эффективно по инструкциям на такт.

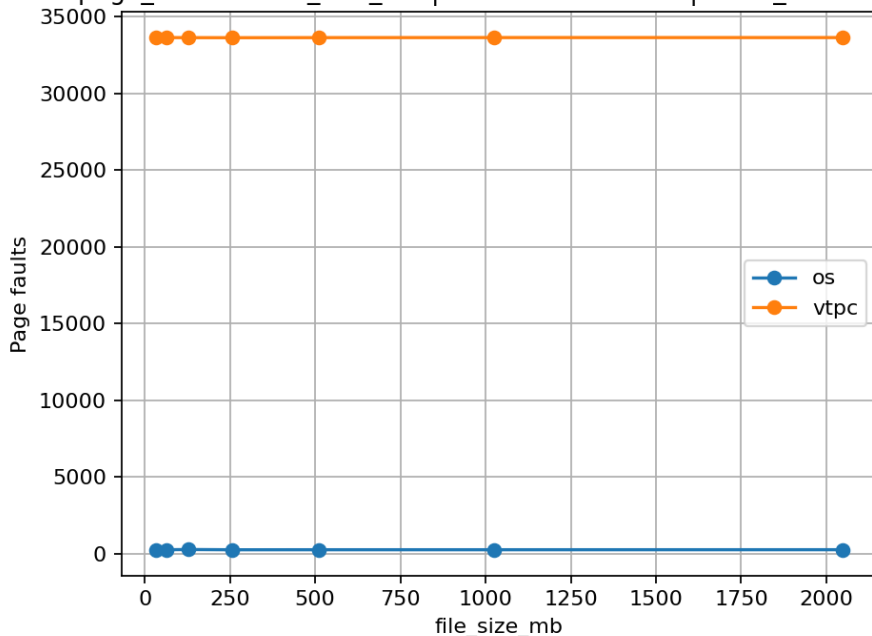
branch_miss_pct vs file_size_mb | direct=off random | block_size=4096



У OS доля промахов предсказателя ветвлений почти постоянна ~9%, у VTPC ниже (~3–5%).

VTPC имеет более предсказуемую логику ветвлений, однако из-за большого числа самих ветвлений его абсолютные потери на ветках всё равно могут быть выше.

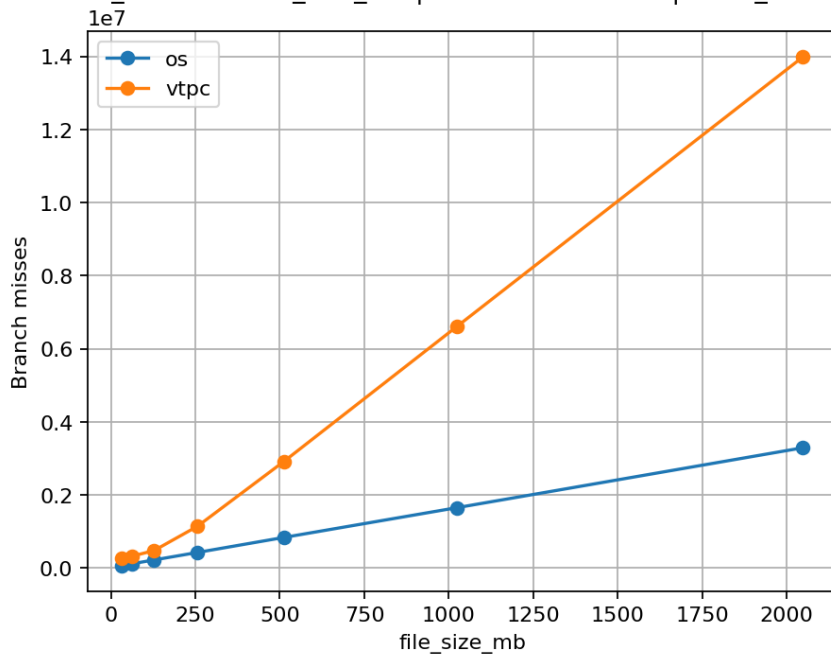
page_faults vs file_size_mb | direct=off random | block_size=4096



У VTPC количество page faults стабильно огромное (~33k), у OS очень маленькое (~250).

Это говорит о большом количестве обращений VTPC к новым страницам памяти (аллокации/буферы/структуры кеша). Это дополнительная нагрузка и объясняет рост накладных расходов VTPC.

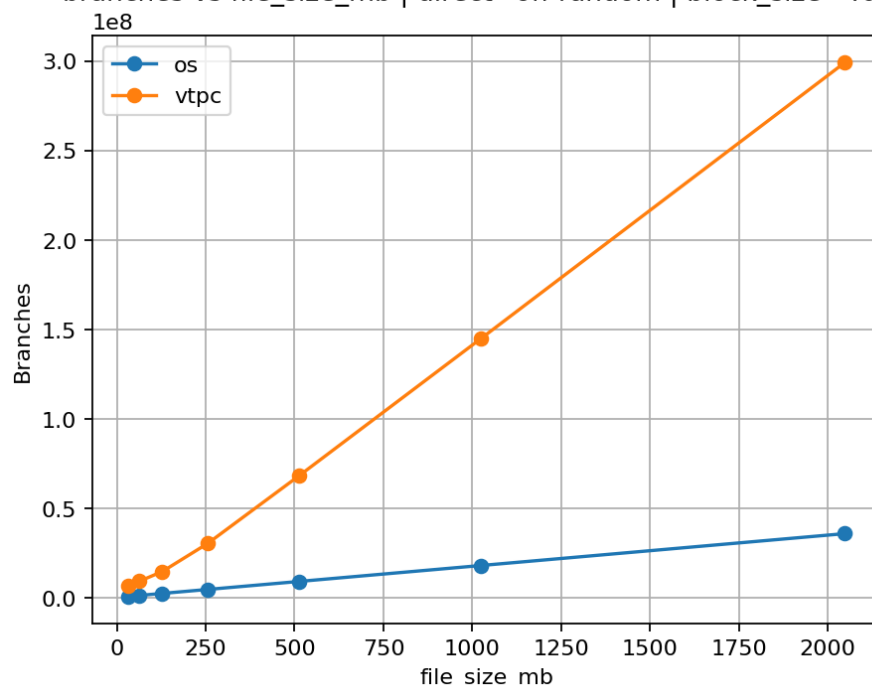
branch_misses vs file_size_mb | direct=off random | block_size=4096



Абсолютное число branch misses у VTPC растёт сильно быстрее, чем у OS.

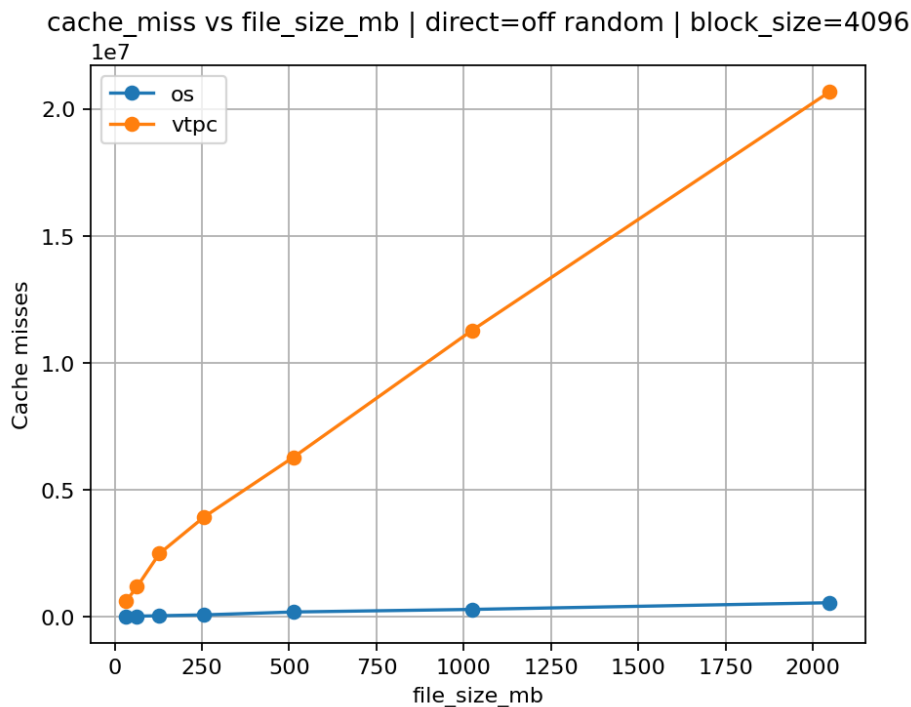
Хотя процент промахов ниже, VTPC выполняет намного больше условной логики → суммарные потери на mispredict'ах становятся заметными и увеличивают CPU overhead.

branches vs file_size_mb | direct=off random | block_size=4096



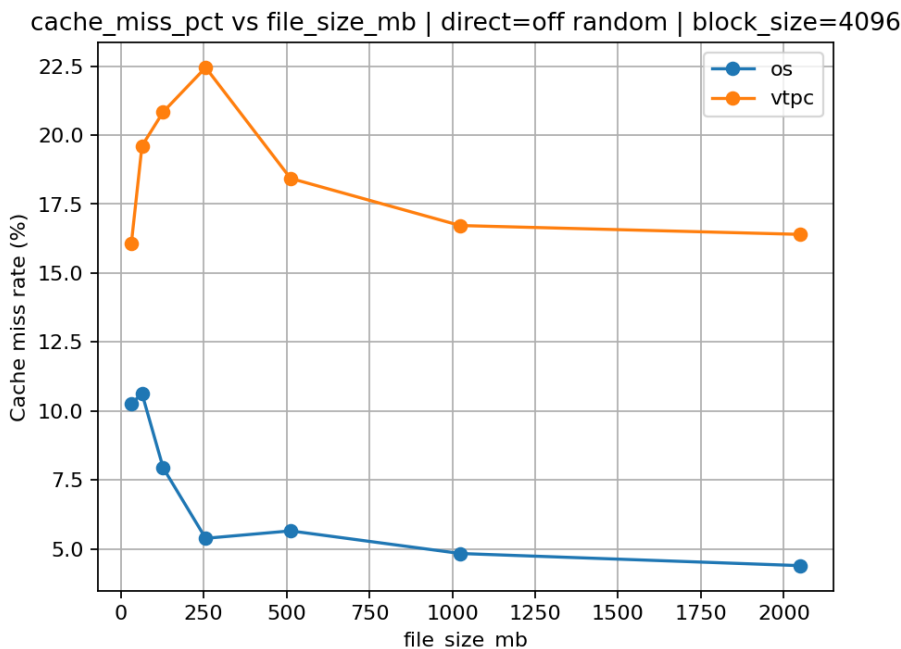
VTPC выполняет существенно больше ветвлений, чем OS, причём рост почти линейный с размером файла.

Это показывает, что VTPC тратит много времени на внутреннюю логику управления кешем (проверки, поиск, обновление LRU и т.д.), тогда как OS делает более “прямой” путь обработки



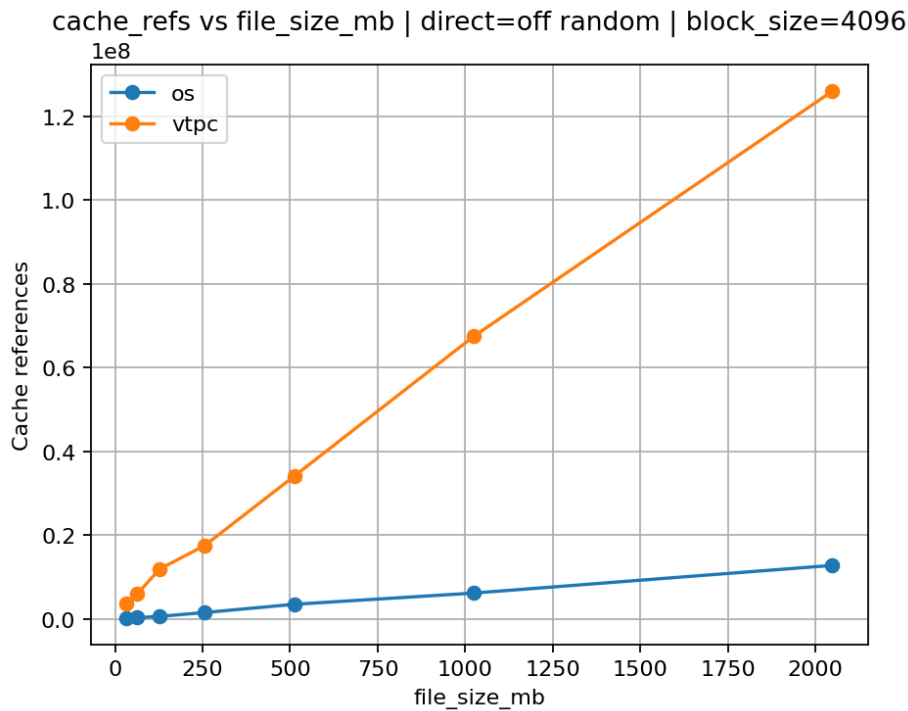
У VTPC cache misses намного больше и растут быстрее, чем у OS.

Это признак плохой локальности доступа к структурам данных VTPC (хэш-таблицы/списки/метаданные), из-за чего CPU чаще ждёт память → падает IPC и растут cycles.

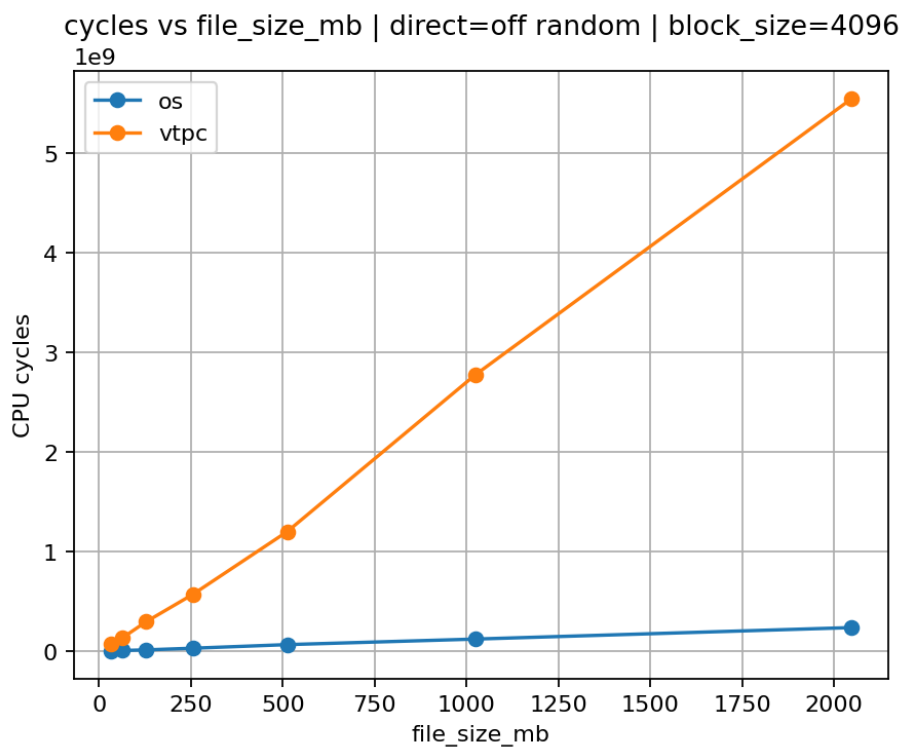


У OS miss rate заметно ниже и в целом снижается на больших файлах (~10% → ~4–5%), а у VTPC держится высоким (~16–22%).

Это означает, что обращения VTPC к данным хуже попадают в CPU cache, поэтому его эффективность по памяти ниже даже при наличии кеширования страниц.

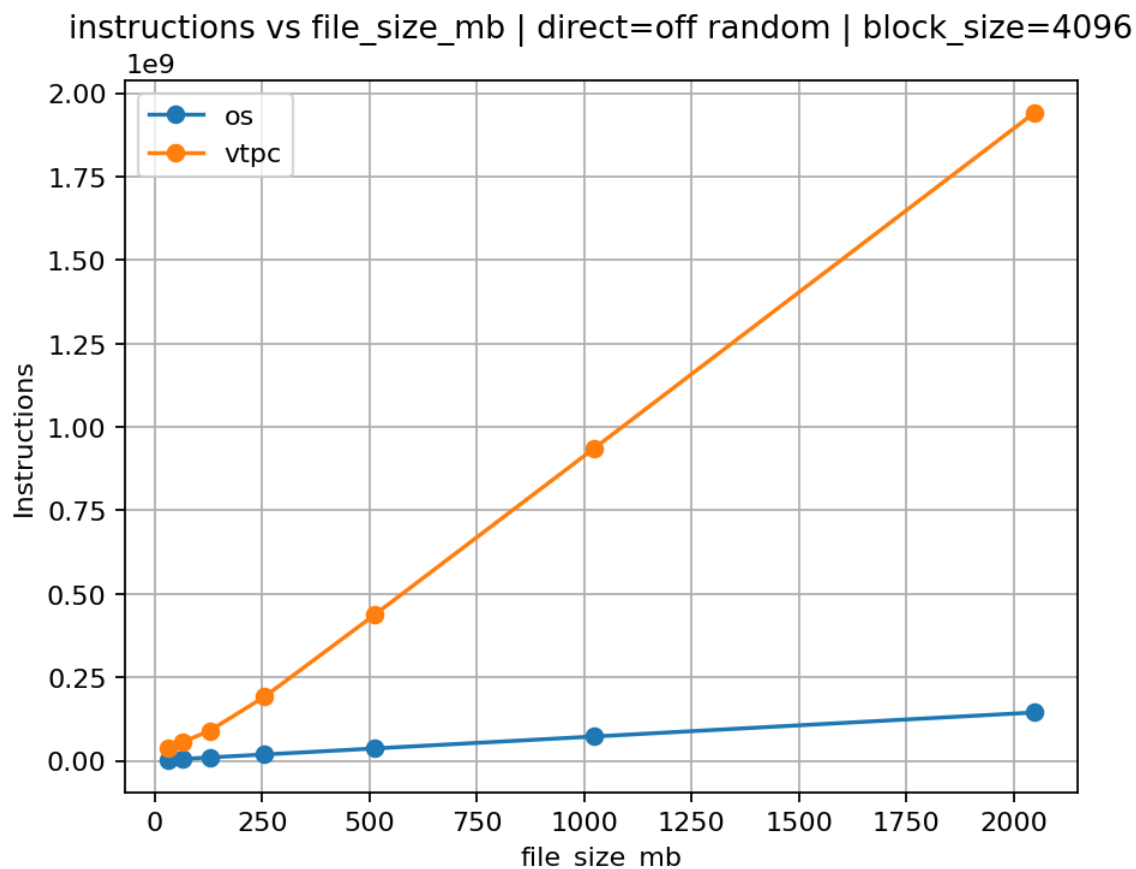


VTPC делает гораздо больше cache references, чем OS, и рост почти линейный. То есть VTPC значительно активнее “трогает” память: больше проверок/операций с метаданными и структурой кеша → выше нагрузка на CPU cache subsystem.



У VTPC число CPU cycles на порядок выше, и увеличение размера файла даёт резкий линейный рост.

Это говорит о большой вычислительной цене обслуживания кеша в user-space: даже при похожем I/O VTPC требует намного больше процессорного времени.



У VTPC количество инструкций гораздо больше и растёт быстрее, чем у OS.

Это подтверждает, что VTPC выполняет много дополнительной работы поверх простого чтения: управление LRU, проверки попаданий, обновления структур и т.п.

Общий вывод по perf:

По perf-графикам (direct=off, random, bs=4096) видно, что **OS эффективнее по CPU**, чем VTPC: у VTPC заметно больше **cycles/instructions** и ниже **IPC**. Также у VTPC выше нагрузка на память — больше **cache refs/misses** и выше **miss rate**, плюс больше ветвлений и **page faults**. Это означает, что VTPC тратит много ресурсов на внутреннюю логику кеша (LRU/метаданные), поэтому на random-доступе обычно проигрывает кешу ОС.

Сбор метрик через eBPF:

Также был реализован сбор метрик через eBPF

На тест - rw=read type=random direct=off bs=4096 bc=524288

Получили вот такие результаты

=== eBPF stats (USDT) ===

hits=22422 misses=1419150 hitrate=1.55%%

Hitrate получили меньше из за того что прогоняли на 1 процессе вместо 4 что были в тестах до.

Общие выводы по работе:

В работе сравнивались два подхода к ускорению чтения данных: стандартный page cache ОС и пользовательский LRU-кэш VTPC.

Эксперименты с разными размерами блоков, объёмами файла и типами доступа показали, что производительность сильно зависит от характера нагрузки.

При последовательном чтении ОС чаще работает стабильнее и быстрее за счёт встроенной оптимизации и предвыборки, а при случайном доступе

VTPC даёт прирост только пока рабочие данные укладываются в кэш (высокий hitrate). Когда размер файла становится значительно больше размера кэша, доля промахов растёт и VTPC начинает заметно проигрывать из-за дополнительных накладных расходов.

Это подтверждает профилирование: у VTPC больше instructions/cycles, ниже IPC и выше нагрузка на память (cache misses, page faults).

В итоге можно сделать вывод, что VTPC полезен в режимах с хорошей повторяемостью обращений и достаточным размером кэша, но в общем случае page cache ОС остаётся более эффективным и универсальным решением.